Examen de première session du 10 décembre 2024
Année universitaire 2024-2025

Parcours : L3 Informatique Générale
Épreuve de : PROG5 (GBIN5U03)
Durée de l'épreuve : 2h
Documents autorisés: Aucun, excepté un dictionnaire bilingue non annoté pour les étudiants étrangers.
Pour chaque question, une partie des points peut tenir compte de la présentation. Le barême est indicatif.

UFR IM²AG

UNIVERSITÉ Grenoble Alpes

1 Compilation (6 points)

Soit un programme dont le code source est organisé en plusieurs fichiers : `pile.c`, `pile.h`, `pile_io.c`, `pile_io.h` et `essai_pile.c`. Le fichier `pile.h` est inclus dans tous les `.c`, tandis que `pile_io.h` est inclus dans `pile_io.c` et `essai_pile.c`. Un étudiant vous a transmis le contenu du `Makefile` qu'il a écrit pour compiler ce programme de manière séparée :

```
CC=clang
CFLAGS=-Wall -Werror -g
FICHIERS=$(wildcard *.c)

essai_pile: $(FICHIERS)
    $(CC) -o $@ $(FICHIERS)

%.o: %.c
    $(CC) $(CFLAGS) -c $^

essai_pile.o: $(patsubst %.c, %.h, $(FICHIERS))
pile_io.o: pile_io.h pile.h
pile.o: pile.h
```

Nous supposons que le répertoire courant ne contient aucun autre fichier que les 6 précédents (les `.h`, `.c` et le `Makefile`).

Questions :

1. (1 point) expliquez en détail ce qu'il se passe lors de l'exécution de la commande `make` (dans le répertoire courant) ?

2. (2,5 points) après la commande exécutée en 1, donnez l'affichage produit par l'exécution de chacune des commandes suivantes (toujours dans le répertoire courant) :

   (a) `make pile_io.o`

   (b) `make essai_pile.o`

   (c) `touch pile_io.h;make essai_pile`

   (d) `touch pile_io.c;make essai_pile`

   (e) `make clean && make`

3. (1,5 points) malheureusement ce `Makefile` ne permet pas d'effectuer la compilation séparée de nos fichiers source pour produire notre programme. Donnez les erreurs contenues dans ce `Makefile` et, pour chacune d'elles le moyen de la corriger ?

4. (1 point) qu'est-ce qui ne va pas dans le fichier `pile.h` suivant :

```
#ifndef __PILE__

typedef struct noeud {
  int donnee;
  struct noeud *prochain;
} *pile;

pile pile_vide() {return NULL}
void detruire_pile(pile p);
pile empiler(int donnee, pile p);
int sommet(pile p) {return p.donnee;}
pile depiler(pile p);
int est_pile_vide(pile p);

#endif
```

2 Tail (14 points)

Dans cette partie, vous aurez probablement besoin de la fonction `getopt` de la bibliothèque standard (`libc`, entêtes dans `unistd.h`), qui facilite les gestion des options dans un programme et dont voici un exemple d'utilisation :

```
// Traitement des options
int opt;
while ((opt = getopt(argc, argv, "x:y")) != -1) {
  switch (opt) {
  case 'x':
    printf("Option x rencontrée, argument : %s\n", optarg);
      break;
  case 'y':
    printf("Option y rencontrée\n");
      break;
  default: /* '?' */
    // option inconnue, getopt affiche un message d'erreur
    exit(42);
  }
}
printf("Autres arguments :\n");
for (int i=optind; i<argc; i++)
  printf("- argument %d : %s\n", i, argv[i]);
```

La fonction `getopt` travaille sur les arguments de la ligne de commande, généralement transmis au `main` sous le nom d'`argc` et `argv`, afin de reconnaitre les arguments qui correspondent à des options. On l'utilise en l'appelant à plusieurs reprises jusqu'à ce qu'elle ait traité toutes les options, auquel cas elle renvoie `-1`. Pour avancer dans son traitement, elle maintient plusieurs variables globales, notamment :

- `optind` : indice de la première option non encore traitée par `getopt` ;
- `optarg` : pointeur sur l'argument d'une option, si celle-ci prend un argument.

L'appel à `getopt`, outre les arguments de la ligne de commande, prend comme paramètre une chaine de caractères décrivant les options qu'elle est censée reconnaître : chaque caractère de la chaine correspond à une option reconnue et, lorsqu'un caractère `x` de cette chaine est suivi du caractère `':'`, cela signifie que l'option `x` prend un argument. En plus de nous simplifier la tâche, `getopt` a l'avantage d'être conforme au standard POSIX : elle accepte les options regroupées en paquets et les arguments collés à leur option.

L'objectif général de cette partie est d'écrire un programme complet ayant le même comportement que la commande `tail`, dont voici un extrait de la page de manuel[1] :

```
NOM

       tail - Afficher la dernière partie de fichiers

SYNOPSIS

       tail [OPTION]... [FICHIER]...

       Afficher  les  10  dernières  lignes  de  chaque  FICHIER sur la sortie
       standard. Lorsqu'il y a plus d'un FICHIER, faire précéder chaque groupe
       de lignes d'un en-tête donnant le nom du fichier. L'entrée standard est
       lue quand FICHIER est omis ou quand FICHIER vaut « - ».

       -c K
          afficher  les  K derniers octets ; vous pouvez aussi utiliser -c
          +K pour afficher les octets de chaque fichier à partir du  Kième
          octet

       -n K
          afficher  les  K dernières lignes, au lieu des 10 dernières ; ou
          utilisez -n +K pour afficher toutes les lignes à  partir  de  la
          Kième.

       -q
          ne jamais afficher les en-têtes donnant les noms de fichiers
```

[1] Rappel : par défaut l'entrée standard correspond au clavier et la sortie standard à l'écran.

Question :

1. Définissez un type `buffer` destiné à stocker une séquence de caractères dont la taille sera dynamique et écrivez les fonctions dont les prototypes sont donnés ci-dessous, permettant respectivement d'allouer un nouveau `buffer`, de libérer un `buffer` précédemment alloué, de connaître la longeur d'un `buffer`, de supprimer un caractère au début d'un `buffer` puis renvoyer sa valeur et d'ajouter un caractère à la fin d'un `buffer` :

   ```
   buffer buffer_alloc();
   void buffer_free(buffer b);
   int buffer_len(buffer b);
   int buffer_pop(buffer b);
   int buffer_append(buffer b, char element);
   ```

   Remarques : pour la plupart, ces fonctions renvoient un `int` même si on n'attend pas de résultat de leur part, cet `int` correspond à un code d'erreur. La fonction `buffer_pop` renvoie un `int`, comme `getchar` de la bibliothèque standard, pour les même raisons.

2. Écrivez une fonction `tail` permettant d'afficher, selon la valeur de l'un de ses paramètres, soit les `x` dernières lignes, soit les `x` derniers caractères lus sur un flux d'entrée `f`, `x` et `f` étant donnés en paramètre à la fonction. Cette fonction devra effectuer son travail en une seule passe de lecture sur le flux d'entrée et aura donc besoin de stocker une partie de ce qui est lu afin de pouvoir produire le bon affichage final. Si cela vous semble plus simple, vous pouvez séparer la fonction `tail` en deux fonctions `tail_lines` et `tail_chars` ;

3. Écrivez un programme ayant le même comportement que la commande `tail`. En cas d'erreur, votre programme devra afficher un message et se terminer avec le code :

   - 1, si l'erreur concerne une mauvaise utilisation des options ;
   - 2, si l'erreur concerne une ouverture de fichier ;
   - 3, si l'erreur concerne une allocation mémoire ;
   - 4, sinon.

   Comme vous l'aurez sans doute remarqué, l'énoncé ne donne aucun guidage pour la gestion des options `-n +K` et `-c +K`, qui seront comptée comme un bonus, à vous d'être créatif !

Le barème tiendra compte de vos efforts pour :

- n'utiliser que la mémoire nécessaire au comportement demandé ;
- ne pas laisser de fuites mémoire, même à la fin du programme ;
- ne pas lire de multiples fois un fichier (le traitement doit se faire en une seule passe) ;
- factoriser votre code.